MATERIA

INF761

Desarrollo de Videojuegos

GUÍA DE LABORATORIO 4

"Space Shooter"

Prefabs, Coroutines y UI básica

Carrera: **Ingeniería Informática**
Docente: **M. Sc. Huáscar Fedor Gonzales Guzmán**
Semestre: **I-2026**
Bloque I: **Fundamentos de Unity**

1. Objetivos de Aprendizaje

Al finalizar esta práctica, el estudiante será capaz de:

- Implementar un sistema de disparo mediante la instanciación de prefabs con dirección y velocidad controladas.
- Utilizar Coroutines (IEnumerator, yield return, StartCoroutine) para controlar el flujo temporal del juego, incluyendo oleadas de enemigos con dificultad progresiva.
- Diseñar una interfaz de usuario funcional con Canvas, TextMeshPro y Canvas Scaler para mostrar puntaje, vidas y pantalla de Game Over.
- Integrar todos los conceptos del Bloque I (física, colisiones, prefabs, scripting) en un mini-juego completo con mecánicas interconectadas.

2. Requisitos Previos

Software	Conocimientos
• Unity 6 LTS (6000.x) • Visual Studio / VS Code con C# • TextMeshPro (importado vía Package Manager)	• Guías 1-3 completadas • GameObjects, componentes, Rigidbody2D • Prefabs e Instantiate • Colisiones y triggers • SceneManager.LoadScene

3. Marco Teórico

3.1 Coroutines en Unity

Las Coroutines son uno de los mecanismos más poderosos de Unity para controlar el flujo temporal de un juego. A diferencia de los métodos convencionales que se ejecutan de principio a fin en un solo frame, una Coroutine puede pausar su ejecución, devolver el control al motor de Unity, y reanudar exactamente donde se detuvo en un frame posterior.

> 📖 **Definición: Coroutine**
>
> Una Coroutine es un método especial que retorna IEnumerator y puede suspender su ejecución mediante instrucciones yield. Unity las gestiona internamente asociándolas al MonoBehaviour que las inicia: si el GameObject se desactiva o destruye, la Coroutine se detiene automáticamente.

La sintaxis básica de una Coroutine sigue este patrón:

```
C#
// Declaración: retorna IEnumerator
IEnumerator SpawnWave()
{
    // Código que se ejecuta inmediatamente
```

```
    Debug.Log("Inicio de oleada");

    // Pausa la ejecución por 2 segundos
    yield return new WaitForSeconds(2f);

    // Continúa después de 2 segundos reales
    Debug.Log("Spawn enemigo");

    // Espera hasta el siguiente frame
    yield return null;

    Debug.Log("Siguiente frame");
}

// Inicio: se llama con StartCoroutine
void Start()
{
    StartCoroutine(SpawnWave());
}
```

Las instrucciones yield más utilizadas son:

Instrucción yield	Comportamiento
yield return null	Espera hasta el siguiente frame
yield return new WaitForSeconds(n)	Pausa n segundos (afectado por Time.timeScale)
yield return new WaitForSecondsRealtime(n)	Pausa n segundos reales (ignora timeScale, ideal para pausas)
yield return new WaitUntil(() => condición)	Espera hasta que la condición sea true
yield return new WaitForFixedUpdate()	Espera hasta el siguiente FixedUpdate
yield return StartCoroutine(otra)	Espera a que otra Coroutine termine (encadenamiento)

> ⚠️ **Precaución**
> Una Coroutine NO es un hilo (thread). Se ejecuta en el hilo principal de Unity y solo "pausa" devolviendo el control al game loop. Esto significa que código pesado dentro de una Coroutine sigue bloqueando el frame. Las Coroutines son ideales para secuencias temporales, no para cálculos intensivos.

Para detener Coroutines en ejecución:

```C#
// Detener una Coroutine específica (guardando referencia)
Coroutine spawnRoutine;
spawnRoutine = StartCoroutine(SpawnWave());
StopCoroutine(spawnRoutine);

// Detener todas las Coroutines del MonoBehaviour
StopAllCoroutines();
```

3.2 Sistema de UI en Unity (Canvas)

Unity utiliza un sistema de UI basado en el componente Canvas, que actúa como contenedor raíz para todos los elementos de interfaz. Cada elemento UI (texto, botones, imágenes) debe ser hijo directo o indirecto de un Canvas para ser visible.

Los componentes fundamentales del sistema UI son:

Componente	Función
Canvas	Contenedor raíz. Define cómo se renderiza la UI (Screen Space - Overlay para HUD, Screen Space - Camera para UI en el mundo).
Canvas Scaler	Controla cómo la UI se adapta a diferentes resoluciones. 'Scale With Screen Size' es el modo recomendado para juegos.
Graphic Raycaster	Permite detectar interacción del mouse/touch con elementos UI (necesario para botones).
EventSystem	GameObject independiente que gestiona los eventos de entrada para la UI. Se crea automáticamente con el primer Canvas.
TextMeshPro (TMP)	Sistema de texto avanzado con renderizado SDF. Reemplaza al Text legacy con mejor calidad y rendimiento.
RectTransform	Variante de Transform para UI. Usa anchors y pivots para posicionamiento relativo y responsive.

💡 **Canvas Scaler: Scale With Screen Size**

Configurar el Canvas Scaler en modo 'Scale With Screen Size' con resolución de referencia 1920x1080 y Match = 0.5 (balance entre ancho y alto) asegura que la UI se vea proporcionalmente correcta en cualquier resolución. Este es el estándar para juegos 2D.

3.3 Prefabs: Instanciación con Dirección

En las guías anteriores utilizamos Instantiate para crear copias de prefabs. En este proyecto profundizamos en dos patrones avanzados: instanciación con velocidad dirigida (para proyectiles) y destrucción temporizada con Destroy(gameObject, tiempo).

```
C#
// Instanciar un proyectil y asignarle velocidad
GameObject bullet = Instantiate(bulletPrefab, firePoint.position,
Quaternion.identity);
bullet.GetComponent<Rigidbody2D>().linearVelocity = Vector2.up * bulletSpeed;

// Destruir automáticamente después de 3 segundos (limpieza de memoria)
Destroy(bullet, 3f);
```

> ⚠️ **Importante: linearVelocity en Unity 6**
>
> A partir de Unity 6, la propiedad 'velocity' de Rigidbody2D fue renombrada a 'linearVelocity'. Si estás usando Unity 6 (6000.x), usa linearVelocity. En versiones anteriores (2022 LTS), usa velocity. Esta guía utiliza linearVelocity para Unity 6.

4. Desarrollo Paso a Paso

En esta sección construiremos el juego Space Shooter de forma incremental. Cada paso produce un resultado verificable.

4.1 Configuración del Proyecto

1. Abrir Unity Hub y crear un nuevo proyecto con la plantilla 2D (Built-In Render Pipeline). Nombre del proyecto: SpaceShooter.
2. Configurar la resolución de la cámara: seleccionar Main Camera en la Hierarchy, en el Inspector verificar que el componente Camera tenga Size = 5 (tamaño ortográfico) y Background = negro (#000000).
3. Crear la estructura de carpetas en el panel Project:
   - **Assets/Scripts** — Scripts C# del juego
   - **Assets/Prefabs** — Prefabs de bala, enemigos, explosiones
   - **Assets/Sprites** — Sprites de la nave, enemigos, balas
   - **Assets/Scenes** — Escena principal del juego
4. Importar TextMeshPro: ir a Window > TextMeshPro > Import TMP Essential Resources, aceptar.

> 💡 **Sprites provisionales**
> Si no tienes sprites de naves, puedes usar sprites geométricos simples creados en Unity: un triángulo blanco para la nave (Right Click > 2D Object > Sprite > Triangle), un cuadrado rojo para enemigos y un círculo amarillo pequeño para las balas. Luego puedes reemplazarlos por sprites artísticos.

4.2 Nave del Jugador: Movimiento

5. En la Hierarchy, crear un nuevo GameObject (Right Click > 2D Object > Sprites > Triangle) y renombrarlo a Player. Ajustar el color (#00FFFF) para visibilidad sobre fondo negro.
6. Agregar un BoxCollider2D (Is Trigger = false) y un Rigidbody2D (Body Type = Kinematic, ya que el jugador se mueve por input, no por física).
7. Crear el script PlayerController.cs en Assets/Scripts y asignarlo al Player:

```
C#
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    [Header("Movimiento")]
    [SerializeField] private float moveSpeed = 7f;

    // Límites de la pantalla (en unidades de mundo)
    private float xMin, xMax, yMin, yMax;
    private float padding = 0.5f;
```

```
    void Start()
    {
        // Calcular límites basados en la cámara
        Camera cam = Camera.main;
        xMin = cam.ViewportToWorldPoint(Vector3.zero).x + padding;
        xMax = cam.ViewportToWorldPoint(Vector3.one).x - padding;
        yMin = cam.ViewportToWorldPoint(Vector3.zero).y + padding;
        yMax = cam.ViewportToWorldPoint(Vector3.one).y - padding;
    }

    void Update()
    {
        float h = Input.GetAxis("Horizontal");
        float v = Input.GetAxis("Vertical");

        Vector3 movement = new Vector3(h, v, 0f) * moveSpeed * Time.deltaTime;
        transform.position += movement;

        // Clamping: restringir posición a los límites de pantalla
        float clampedX = Mathf.Clamp(transform.position.x, xMin, xMax);
        float clampedY = Mathf.Clamp(transform.position.y, yMin, yMax);
        transform.position = new Vector3(clampedX, clampedY, 0f);
    }
}
```

📖 ¿Qué es ViewportToWorldPoint?

El método Camera.ViewportToWorldPoint convierte coordenadas de viewport (0,0 = esquina inferior izquierda; 1,1 = esquina superior derecha) a coordenadas del mundo. Esto nos permite calcular los límites de pantalla sin importar el tamaño de la cámara, haciendo el código adaptable a cualquier resolución.

8. Probar el juego: la nave debe moverse con las flechas o WASD y detenerse en los bordes de la pantalla.

4.3 Sistema de Disparo

9. Crear el prefab de la bala: en la Hierarchy, crear un GameObject (Right Click > 2D Object > Sprites > Circle) llamado Bullet. Ajusta el tamaño en Scale X=0.3, Y=0.3. Position Y=1. Asignar color amarillo #FFFF00. Agregar un CircleCollider2D (Is Trigger = true) y un Rigidbody2D (Body Type = Dynamic, Gravity Scale = 0).
10. Crear el script Bullet.cs y asignarlo al GameObject Bullet:

```
C#
using UnityEngine;

public class Bullet : MonoBehaviour
{
    [SerializeField] private float speed = 12f;

    void Start()
    {
        // Asignar velocidad hacia arriba al instanciarse
        GetComponent<Rigidbody2D>().linearVelocity = Vector2.up * speed;
```

```
        // Auto-destruir después de 3 segundos (limpieza)
        Destroy(gameObject, 3f);
    }
}
```

11. Arrastrar el GameObject Bullet desde la Hierarchy al panel Project (carpeta Prefabs) para convertirlo en Prefab. Luego eliminar la instancia de la Hierarchy.
12. Crear un punto de disparo: en el Player, agregar un GameObject hijo vacío llamado FirePoint y posicionarlo ligeramente por encima de la nave (Y = 0.7 aproximadamente).
13. Actualizar PlayerController.cs para agregar el sistema de disparo:

```
C#
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    [Header("Movimiento")]
    [SerializeField] private float moveSpeed = 7f;

    [Header("Disparo")]
    [SerializeField] private GameObject bulletPrefab;
    [SerializeField] private Transform firePoint;
    [SerializeField] private float fireRate = 0.3f;
    private float nextFireTime = 0f;

    private float xMin, xMax, yMin, yMax;
    private float padding = 0.5f;

    void Start()
    {
        Camera cam = Camera.main;
        xMin = cam.ViewportToWorldPoint(Vector3.zero).x + padding;
        xMax = cam.ViewportToWorldPoint(Vector3.one).x - padding;
        yMin = cam.ViewportToWorldPoint(Vector3.zero).y + padding;
        yMax = cam.ViewportToWorldPoint(Vector3.one).y - padding;
    }

    void Update()
    {
        Move();
        Shoot();
    }

    void Move()
    {
        float h = Input.GetAxis("Horizontal");
        float v = Input.GetAxis("Vertical");
        Vector3 movement = new Vector3(h, v, 0f) * moveSpeed * Time.deltaTime;
        transform.position += movement;

        float clampedX = Mathf.Clamp(transform.position.x, xMin, xMax);
        float clampedY = Mathf.Clamp(transform.position.y, yMin, yMax);
        transform.position = new Vector3(clampedX, clampedY, 0f);
    }

    void Shoot()
    {
        // Disparar al presionar Espacio, respetando el fireRate
```

```
        if (Input.GetKey(KeyCode.Space) && Time.time >= nextFireTime)
        {
            nextFireTime = Time.time + fireRate;
            Instantiate(bulletPrefab, firePoint.position, Quaternion.identity);
        }
    }
}
```

14. En el Inspector del Player, asignar el prefab Bullet al campo bulletPrefab y el hijo FirePoint al campo firePoint.
15. Probar: al presionar Espacio, la nave debe disparar balas amarillas hacia arriba.

4.4 Creación de Enemigos

16. Crear el prefab del enemigo: GameObject (Right Click > 2D Object > Sprites > Square) vacío llamado Enemy. Asignar color rojo #FF4444, BoxCollider2D (Is Trigger = true), Rigidbody2D (Dynamic, Gravity Scale = 0).
17. Crear el script Enemy.cs y asignarlo al GameObject Enemy.

```
C#
using UnityEngine;

public class Enemy : MonoBehaviour
{
    [SerializeField] private float speed = 3f;
    [SerializeField] private int scoreValue = 100;

    void Start()
    {
        // Movimiento constante hacia abajo
        GetComponent<Rigidbody2D>().linearVelocity = Vector2.down * speed;
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Bullet"))
        {
            // Bala impacta enemigo: destruir ambos y sumar puntaje
            GameManager.Instance.AddScore(scoreValue);
            Destroy(other.gameObject);
            Destroy(gameObject);
        }
        else if (other.CompareTag("Player"))
        {
            // Enemigo colisiona con jugador: aplicar daño
            GameManager.Instance.PlayerHit();
            Destroy(gameObject);
        }
    }

    // Destruir si sale de la pantalla (por abajo)
    void OnBecameInvisible()
    {
        Destroy(gameObject);
    }
}
```

18. Convertir en Prefab: arrastrar Enemy al panel Project (carpeta Prefabs). Eliminar de la Hierarchy.
19. Configurar Tags: ir a Edit > Project Settings > Tags and Layers. Crear los tags Bullet y Player si no existen. Asignar el tag Player al GameObject Player y el tag Bullet al prefab Bullet.

> ⚠️ **Importante: Tags para colisiones**
> El método CompareTag es más eficiente y seguro que comparar con == contra la propiedad tag. Siempre usa CompareTag en lugar de gameObject.tag == "nombre". Además, asegúrate de que los tags estén creados ANTES de asignarlos.

4.5 Spawner de Enemigos con Coroutines

Aquí aplicamos los conceptos teóricos de Coroutines para crear un sistema de oleadas con dificultad progresiva.

20. Crear un GameObject vacío llamado EnemySpawner y posicionarlo fuera de la vista de la cámara (Y = 6 aproximadamente, por encima de la pantalla).
21. Crear el script EnemySpawner.cs y asignarlo al GameObject EnemySpawner.

```C#
using UnityEngine;
using System.Collections;

public class EnemySpawner : MonoBehaviour
{
    [Header("Configuración de Spawn")]
    [SerializeField] private GameObject enemyPrefab;
    [SerializeField] private float spawnInterval = 1.5f;
    [SerializeField] private float minSpawnInterval = 0.4f;
    [SerializeField] private float difficultyIncreaseRate = 0.05f;

    [Header("Oleadas")]
    [SerializeField] private int enemiesPerWave = 5;
    [SerializeField] private int extraEnemiesPerWave = 2;
    [SerializeField] private float timeBetweenWaves = 3f;

    private float xMin, xMax;
    private int currentWave = 0;

    void Start()
    {
        Camera cam = Camera.main;
        xMin = cam.ViewportToWorldPoint(Vector3.zero).x + 0.5f;
        xMax = cam.ViewportToWorldPoint(Vector3.one).x - 0.5f;

        StartCoroutine(SpawnWaves());
    }

    IEnumerator SpawnWaves()
    {
        // Esperar 2 segundos antes de la primera oleada
        yield return new WaitForSeconds(2f);

        while (true) // Oleadas infinitas
```

```
        {
            currentWave++;
            int enemiesToSpawn = enemiesPerWave + (currentWave - 1) *
extraEnemiesPerWave;

            // Calcular intervalo actual (más rápido cada oleada)
            float currentInterval = Mathf.Max(
                spawnInterval - (currentWave - 1) * difficultyIncreaseRate,
                minSpawnInterval
            );

            Debug.Log($"=== Oleada {currentWave}: {enemiesToSpawn} enemigos ===");

            for (int i = 0; i < enemiesToSpawn; i++)
            {
                SpawnEnemy();
                yield return new WaitForSeconds(currentInterval);
            }

            // Pausa entre oleadas
            yield return new WaitForSeconds(timeBetweenWaves);
        }
    }

    void SpawnEnemy()
    {
        float randomX = Random.Range(xMin, xMax);
        Vector3 spawnPos = new Vector3(randomX, transform.position.y, 0f);
        Instantiate(enemyPrefab, spawnPos, Quaternion.identity);
    }
}
```

📖 Análisis de la Coroutine SpawnWaves

Observa cómo la Coroutine SpawnWaves usa un bucle while(true) que nunca termina. Esto es seguro porque el yield return dentro del bucle devuelve el control a Unity en cada iteración. Sin el yield, sería un bucle infinito que congelaría el juego. La Coroutine se detendrá automáticamente si el EnemySpawner es destruido.

22. Asignar el prefab Enemy al campo enemyPrefab en el Inspector del EnemySpawner.
23. Probar: los enemigos deben aparecer desde arriba con posición X aleatoria y moverse hacia abajo. Cada oleada debe tener más enemigos y menos tiempo entre spawns.

4.6 GameManager: Puntaje, Vidas y Flujo del Juego

El GameManager centraliza la lógica del juego usando el patrón Singleton para acceso global.

24. Crear un GameObject vacío llamado GameManager en la Hierarchy.
25. Crear el script GameManager.cs:

```C#
using UnityEngine;
using UnityEngine.SceneManagement;
using TMPro;
```

```
public class GameManager : MonoBehaviour
{
    // ── Singleton ──────────────────────────────────────
    public static GameManager Instance { get; private set; }

    [Header("Configuración")]
    [SerializeField] private int maxLives = 3;

    [Header("Referencias UI")]
    [SerializeField] private TextMeshProUGUI scoreText;
    [SerializeField] private TextMeshProUGUI livesText;
    [SerializeField] private GameObject gameOverPanel;

    private int score = 0;
    private int lives;
    private bool isGameOver = false;

    void Awake()
    {
        // Patrón Singleton: asegurar una sola instancia
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
    }

    void Start()
    {
        lives = maxLives;
        UpdateUI();
        gameOverPanel.SetActive(false);
    }

    public void AddScore(int value)
    {
        if (isGameOver) return;
        score += value;
        UpdateUI();
    }

    public void PlayerHit()
    {
        if (isGameOver) return;

        lives--;
        UpdateUI();

        if (lives <= 0)
        {
            GameOver();
        }
        else
        {
            // Efecto visual de daño (flash rojo)
            StartCoroutine(DamageFlash());
        }
    }
```

```
    System.Collections.IEnumerator DamageFlash()
    {
        SpriteRenderer sr = FindAnyObjectByType<PlayerController>()
            .GetComponent<SpriteRenderer>();
        Color originalColor = sr.color;

        // Flash rojo
        sr.color = Color.red;
        yield return new WaitForSeconds(0.15f);
        sr.color = originalColor;
        yield return new WaitForSeconds(0.1f);
        sr.color = Color.red;
        yield return new WaitForSeconds(0.15f);
        sr.color = originalColor;
    }

    void UpdateUI()
    {
        scoreText.text = $"Puntaje: {score}";
        livesText.text = $"Vidas: {lives}";
    }

    void GameOver()
    {
        isGameOver = true;
        gameOverPanel.SetActive(true);

        // Detener el spawner
        EnemySpawner spawner = FindAnyObjectByType<EnemySpawner>();
        if (spawner != null) spawner.StopAllCoroutines();

        // Opcional: pausar el tiempo del juego
        Time.timeScale = 0f;
    }

    // Llamado por el botón de reinicio
    public void RestartGame()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
    }
}
```

📖 **Patrón Singleton**

El patrón Singleton garantiza que exista una única instancia de GameManager accesible desde cualquier script mediante GameManager.Instance. Esto evita buscar referencias manualmente. Nota: en juegos grandes se prefieren alternativas como ScriptableObjects o Service Locator, pero para proyectos pequeños el Singleton es práctico y directo.

4.7 Interfaz de Usuario con Canvas

26. Crear el Canvas: Right Click en la Hierarchy > UI > Canvas. Verificar que se cree también un EventSystem.
27. Configurar el Canvas Scaler (componente del Canvas):
    - **UI Scale Mode:** Scale With Screen Size

- **Reference Resolution:** 1920 x 1080
- **Match:** 0.5 (balance entre ancho y alto)

28. Crear el texto de puntaje: Right Click sobre Canvas > UI > Text - TextMeshPro. Renombrar a ScoreText.
    - **Rect Transform:** Anchor = Top Left, Pos X = 20, Pos Y = -20, Width = 300, Height = 50
    - **Font Size:** 36
    - **Text:** "Puntaje: 0"
    - **Color:** Blanco

29. Crear el texto de vidas: duplicar ScoreText (Ctrl+D), renombrar a LivesText.
    - **Anchor:** Top Right, Pos X = -20, Pos Y = -20
    - **Alignment:** Right
    - **Text:** "Vidas: 3"

30. Crear el panel de Game Over:
    - Right Click sobre Canvas > UI > Panel. Renombrar a GameOverPanel.
    - **Rect Transform:** Stretch en ambos ejes (llenar toda la pantalla)
    - **Image Color:** Negro con Alpha = 180 (semi-transparente)

31. Agregar contenido al GameOverPanel:
    - Hijo TextMeshPro: texto "GAME OVER", Font Size = 72, color rojo, centrado.
    - Hijo Button - TextMeshPro: texto "Reiniciar", Font Size = 36.

32. Configurar el botón de reinicio: seleccionar el Button, en el Inspector > On Click() agregar una entrada. Arrastrar el GameManager al campo del objeto, seleccionar la función GameManager > RestartGame().

> ⚠️ **Cuidado con Time.timeScale = 0**
> Al pausar el juego con Time.timeScale = 0, los botones UI siguen funcionando porque el EventSystem no depende del timeScale. Sin embargo, cualquier WaitForSeconds en Coroutines se detendrá. Si necesitas Coroutines durante la pausa, usa WaitForSecondsRealtime en su lugar.

33. Conectar las referencias UI en el GameManager: en el Inspector del GameManager, arrastrar ScoreText al campo scoreText, LivesText al campo livesText, y GameOverPanel al campo gameOverPanel.

4.8 Configuración Final de Colisiones

Para que el sistema de colisiones funcione correctamente, necesitamos asegurar la configuración de los colliders y tags:

GameObject	Collider	Is Trigger	Tag
Player	BoxCollider2D	✓ (true)	Player
Bullet	CircleCollider2D	✓ (true)	Bullet
Enemy	BoxCollider2D	✓ (true)	Enemy

📖 **¿Por qué todos son Triggers?**
En este juego usamos triggers (Is Trigger = true) en lugar de colisiones físicas porque no queremos que los objetos reboten o se empujen entre sí. Los triggers detectan la superposición sin generar respuesta física, lo cual es perfecto para un shooter donde solo necesitamos saber cuándo dos objetos se tocan para aplicar lógica (destruir, dañar, puntuar).

Asegurarse de que el BoxCollider2D del Player tenga Is Trigger = true. Actualizar el script Enemy.cs para que OnTriggerEnter2D detecte tanto balas como el jugador (ya incluido en el código anterior).

4.9 Pruebas y Verificación

Antes de continuar, verifica que todos los sistemas funcionen:

	Verificación
✓	La nave se mueve con flechas/WASD y no sale de la pantalla
✓	Al presionar Espacio, las balas salen desde el punto de disparo hacia arriba
✓	Las balas se destruyen automáticamente después de 3 segundos
✓	Los enemigos aparecen desde arriba con posición X aleatoria
✓	Las oleadas aumentan en cantidad de enemigos y frecuencia de aparición
✓	Bala + Enemigo: ambos se destruyen y el puntaje aumenta
✓	Enemigo + Jugador: el enemigo se destruye y las vidas disminuyen
✓	Al perder la última vida, aparece la pantalla de Game Over
✓	El botón Reiniciar recarga la escena correctamente
✓	El flash rojo de daño funciona al recibir impacto

5. Código Fuente Completo

A continuación se presentan las versiones finales de cada script. Copiar estos scripts exactamente como están y verificar las conexiones en el Inspector.

5.1 PlayerController.cs (versión final)

```C#
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    [Header("Movimiento")]
    [SerializeField] private float moveSpeed = 7f;

    [Header("Disparo")]
    [SerializeField] private GameObject bulletPrefab;
    [SerializeField] private Transform firePoint;
    [SerializeField] private float fireRate = 0.3f;
    private float nextFireTime = 0f;

    // Límites de pantalla
    private float xMin, xMax, yMin, yMax;
    private float padding = 0.5f;

    void Start()
    {
        Camera cam = Camera.main;
        xMin = cam.ViewportToWorldPoint(Vector3.zero).x + padding;
        xMax = cam.ViewportToWorldPoint(Vector3.one).x - padding;
        yMin = cam.ViewportToWorldPoint(Vector3.zero).y + padding;
        yMax = cam.ViewportToWorldPoint(Vector3.one).y - padding;
    }

    void Update()
    {
        Move();
        Shoot();
    }

    void Move()
    {
        float h = Input.GetAxis("Horizontal");
        float v = Input.GetAxis("Vertical");
        Vector3 movement = new Vector3(h, v, 0f) * moveSpeed * Time.deltaTime;
        transform.position += movement;

        float cx = Mathf.Clamp(transform.position.x, xMin, xMax);
        float cy = Mathf.Clamp(transform.position.y, yMin, yMax);
        transform.position = new Vector3(cx, cy, 0f);
    }

    void Shoot()
    {
        if (Input.GetKey(KeyCode.Space) && Time.time >= nextFireTime)
        {
            nextFireTime = Time.time + fireRate;
```

```
            Instantiate(bulletPrefab, firePoint.position, Quaternion.identity);
        }
    }
}
```

5.2 Bullet.cs

```C#
using UnityEngine;

public class Bullet : MonoBehaviour
{
    [SerializeField] private float speed = 12f;

    void Start()
    {
        GetComponent<Rigidbody2D>().linearVelocity = Vector2.up * speed;
        Destroy(gameObject, 3f);
    }
}
```

5.3 Enemy.cs

```C#
using UnityEngine;

public class Enemy : MonoBehaviour
{
    [SerializeField] private float speed = 3f;
    [SerializeField] private int scoreValue = 100;

    void Start()
    {
        GetComponent<Rigidbody2D>().linearVelocity = Vector2.down * speed;
    }

    void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Bullet"))
        {
            GameManager.Instance.AddScore(scoreValue);
            Destroy(other.gameObject);
            Destroy(gameObject);
        }
        else if (other.CompareTag("Player"))
        {
            GameManager.Instance.PlayerHit();
            Destroy(gameObject);
        }
    }

    void OnBecameInvisible()
    {
        Destroy(gameObject);
    }
}
```

5.4 EnemySpawner.cs

```
C#
using UnityEngine;
using System.Collections;

public class EnemySpawner : MonoBehaviour
{
    [Header("Spawn")]
    [SerializeField] private GameObject enemyPrefab;
    [SerializeField] private float spawnInterval = 1.5f;
    [SerializeField] private float minSpawnInterval = 0.4f;
    [SerializeField] private float difficultyIncreaseRate = 0.05f;

    [Header("Oleadas")]
    [SerializeField] private int enemiesPerWave = 5;
    [SerializeField] private int extraEnemiesPerWave = 2;
    [SerializeField] private float timeBetweenWaves = 3f;

    private float xMin, xMax;
    private int currentWave = 0;

    void Start()
    {
        Camera cam = Camera.main;
        xMin = cam.ViewportToWorldPoint(Vector3.zero).x + 0.5f;
        xMax = cam.ViewportToWorldPoint(Vector3.one).x - 0.5f;
        StartCoroutine(SpawnWaves());
    }

    IEnumerator SpawnWaves()
    {
        yield return new WaitForSeconds(2f);

        while (true)
        {
            currentWave++;
            int count = enemiesPerWave + (currentWave - 1) * extraEnemiesPerWave;
            float interval = Mathf.Max(
                spawnInterval - (currentWave - 1) * difficultyIncreaseRate,
                minSpawnInterval
            );

            for (int i = 0; i < count; i++)
            {
                SpawnEnemy();
                yield return new WaitForSeconds(interval);
            }

            yield return new WaitForSeconds(timeBetweenWaves);
        }
    }

    void SpawnEnemy()
    {
        float rx = Random.Range(xMin, xMax);
        Vector3 pos = new Vector3(rx, transform.position.y, 0f);
        Instantiate(enemyPrefab, pos, Quaternion.identity);
    }
}
```

5.5 GameManager.cs

```
C#
using UnityEngine;
using UnityEngine.SceneManagement;
using TMPro;
using System.Collections;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }

    [Header("Configuración")]
    [SerializeField] private int maxLives = 3;

    [Header("UI")]
    [SerializeField] private TextMeshProUGUI scoreText;
    [SerializeField] private TextMeshProUGUI livesText;
    [SerializeField] private GameObject gameOverPanel;

    private int score = 0;
    private int lives;
    private bool isGameOver = false;

    void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
    }

    void Start()
    {
        lives = maxLives;
        UpdateUI();
        gameOverPanel.SetActive(false);
    }

    public void AddScore(int value)
    {
        if (isGameOver) return;
        score += value;
        UpdateUI();
    }

    public void PlayerHit()
    {
        if (isGameOver) return;
        lives--;
        UpdateUI();

        if (lives <= 0)
            GameOver();
        else
            StartCoroutine(DamageFlash());
    }
```

```
    IEnumerator DamageFlash()
    {
        var sr = FindAnyObjectByType<PlayerController>()
            .GetComponent<SpriteRenderer>();
        Color original = sr.color;

        sr.color = Color.red;
        yield return new WaitForSeconds(0.15f);
        sr.color = original;
        yield return new WaitForSeconds(0.1f);
        sr.color = Color.red;
        yield return new WaitForSeconds(0.15f);
        sr.color = original;
    }

    void UpdateUI()
    {
        scoreText.text = $"Puntaje: {score}";
        livesText.text = $"Vidas: {lives}";
    }

    void GameOver()
    {
        isGameOver = true;
        gameOverPanel.SetActive(true);

        var spawner = FindAnyObjectByType<EnemySpawner>();
        if (spawner != null) spawner.StopAllCoroutines();

        Time.timeScale = 0f;
    }

    public void RestartGame()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
    }
}
```

6. Ejercicios Propuestos

Los siguientes ejercicios amplían las mecánicas del Space Shooter. Cada uno introduce conceptos nuevos que preparan al estudiante para los proyectos del Bloque II.

Ejercicio 1: Power-Up de Escudo

Implementar un power-up que otorgue un escudo temporal al jugador. Mientras el escudo está activo, el jugador no pierde vidas al recibir impactos.

Requisitos técnicos:

- Crear un prefab ShieldPowerUp que caiga desde arriba (como un enemigo) con un sprite distintivo (color cian, forma de diamante).
- Al recogerlo (OnTriggerEnter2D con tag Player), activar un escudo visual (SpriteRenderer hijo del Player con un círculo semi-transparente).
- El escudo dura 5 segundos. Usar una Coroutine con WaitForSeconds para desactivarlo.
- Mientras el escudo está activo, la variable isShielded = true impide que PlayerHit() descuente vidas.
- El spawner genera un ShieldPowerUp cada 15-20 segundos (usar otra Coroutine independiente).

> 💡 **Pista de implementación**
> Agrega un campo bool isShielded al GameManager y un método ActivateShield() que inicie una Coroutine: active el escudo visual, espere 5 segundos, y lo desactive. En PlayerHit(), revisa if (isShielded) return; antes de descontar vidas.

Ejercicio 2: Disparo Triple

Implementar un power-up que temporalmente cambie el disparo simple por un disparo triple (tres balas simultáneas en abanico).

Requisitos técnicos:

- Crear un prefab TripleShotPowerUp con sprite distintivo (color naranja).
- Al recogerlo, el PlayerController cambia a modo de disparo triple durante 8 segundos.
- El disparo triple instancia 3 balas: una hacia arriba (0°), una a +15° y otra a -15°.
- Usar Quaternion.Euler(0, 0, ángulo) para rotar la dirección de las balas laterales.
- Modificar el script Bullet para que la velocidad siga la dirección del transform: transform.up * speed.

```
C#
// Ejemplo de disparo con ángulo
void ShootTriple()
{
    // Bala central (recto hacia arriba)
    Instantiate(bulletPrefab, firePoint.position, Quaternion.identity);
```

```
    // Bala izquierda (rotada +15°)
    Instantiate(bulletPrefab, firePoint.position, Quaternion.Euler(0, 0, 15f));

    // Bala derecha (rotada -15°)
    Instantiate(bulletPrefab, firePoint.position, Quaternion.Euler(0, 0, -15f));
}

// En Bullet.cs, cambiar Start():
void Start()
{
    GetComponent<Rigidbody2D>().linearVelocity = transform.up * speed;
    Destroy(gameObject, 3f);
}
```

Ejercicio 3: Enemigos con Movimiento Sinusoidal

Crear un nuevo tipo de enemigo que se mueva en un patrón sinusoidal (zigzag) mientras desciende, haciéndolo más difícil de acertar.

Requisitos técnicos:

- Crear un prefab SineEnemy con sprite diferente (color magenta, forma de diamante).
- El movimiento combina descenso constante con oscilación horizontal usando Mathf.Sin.
- El SineEnemy vale 200 puntos (el doble que el enemigo normal).
- El spawner genera SineEnemies a partir de la oleada 3 con probabilidad del 30%.

```C#
using UnityEngine;

public class SineEnemy : MonoBehaviour
{
    [SerializeField] private float fallSpeed = 2.5f;
    [SerializeField] private float waveAmplitude = 2f;   // Amplitud del zigzag
    [SerializeField] private float waveFrequency = 3f;    // Velocidad del zigzag
    [SerializeField] private int scoreValue = 200;

    private float startX;
    private float elapsedTime = 0f;

    void Start()
    {
        startX = transform.position.x;
    }

    void Update()
    {
        elapsedTime += Time.deltaTime;

        // Movimiento sinusoidal
        float newX = startX + Mathf.Sin(elapsedTime * waveFrequency) *
waveAmplitude;
        float newY = transform.position.y - fallSpeed * Time.deltaTime;
        transform.position = new Vector3(newX, newY, 0f);
    }
```

```
    void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Bullet"))
        {
            GameManager.Instance.AddScore(scoreValue);
            Destroy(other.gameObject);
            Destroy(gameObject);
        }
        else if (other.CompareTag("Player"))
        {
            GameManager.Instance.PlayerHit();
            Destroy(gameObject);
        }
    }

    void OnBecameInvisible()
    {
        Destroy(gameObject);
    }
}
```

7. Bibliografía y Recursos

Unity Technologies (2024). *Coroutines - Unity Manual*. Disponible en: https://docs.unity3d.com/Manual/Coroutines.html

Unity Technologies (2024). *UIElements & UI Toolkit - Unity Manual*. Disponible en: https://docs.unity3d.com/Manual/UIToolkits.html

Unity Technologies (2024). *TextMesh Pro Documentation*. Disponible en: https://docs.unity3d.com/Packages/com.unity.textmeshpro@latest

Unity Technologies (2024). *Canvas Scaler - Unity Manual*. Disponible en: https://docs.unity3d.com/Manual/script-CanvasScaler.html

Hocking, J. (2022). *Unity in Action: Multiplatform Game Development in C#*. Manning Publications, 3ra Edición

Ferrone, H. (2023). *Learning C# by Developing Games with Unity*. Packt Publishing, 8va Edición